1. We color the source vertex red;
2. Once we extract a vertex from the queue, we color all its neighbors with the opposite color (say blue if current vertex is red, and vice versa);
3. If any of the adjacent vertices is already colored with the same color as current vertex, then the graph is not bipartite.

Listing 15.6 shows the pseudocode for a method returning the two partitions while it checks if the graph is bipartite; you can also take a look at a Java implementation on the book's repo, or a JavaScript implementation provided by JsGraph, the library that has been used to draw most of the examples of embeddings in this chapter.

15.2.4 Improving performance

This is the bulk of the simplest algorithm we have to test graph planarity; we know it's inefficient... but exactly how inefficient?

It's a recursive algorithm, where from each call to the method can stem several recursive calls; the depth of the recursion is already linear in the size of the original graph `G=(V,E)`, because we remove one vertex or one edge at the time.

The breadth of the recursion tree (the number of recursive calls) is also linear, although in the size of the graph that is currently run on, let's call it `G'=(V', E')`. This is because the two for loops cycle through all vertices and all edges in `G'`.